Справочник по коду

Блочное LU-разложение · Алгоритм 37 пособия Головашкина

$$A' = A_{22} - L_{21}\,U_{12}$$

stage1_seq.c · stage2_vec.c · stage3_omp.c · stage4_mpi.c · скрипты

Intel Core i5-1135G7 · MinGW GCC 9.2 · MS-MPI

Содержание

Справочник по коду

1. Карта репозитория
2. Общие соглашения
3. Одна итерация: от формул к коду
4. Этап 1 — stage1_seq.c
5. Этап 2 — stage2_vec.c
6. Этап 3 — stage3_omp.c
7. Этап 4 — stage4_mpi.c
8. Сравнение реализаций
9. Вспомогательные скрипты
10. Вывод программ
11. Частые вопросы
12. Шпаргалка: где что искать

Справочник по коду

Путеводитель по исходникам проекта. Реализован Алгоритм 37 из параграфа 2.4 пособия Головашкина. Здесь описано, что лежит в каждом файле, какой строке кода соответствует каждая формула алгоритма, где хранятся данные и почему код написан именно так. Математика разложения изложена в 00_lu_theory.md, анализ и результаты каждого этапа — в 01_sequential.md, 02_vectorization.md, 03_openmp.md и 04_mpi.md.

> Листинги в PDF при сборке берутся прямо из исходников, номера строк в них настоящие. Подпись над листингом — ссылка на этот же фрагмент на GitHub.

1. Карта репозитория

Файл	Что внутри
stage1_seq.c	этап 1: последовательный алгоритм, запись через макрос `A(i, j)`
stage2_vec.c	этап 2: те же шаги через функцию `saxpy` с `restrict`; файл собирается дважды
stage3_omp.c	этап 3: код этапа 2 плюс `#pragma omp parallel for`
stage4_mpi.c	этап 4: MPI, блочные строки циклически распределены по процессам
include/sal.h	пустая заглушка: заголовки MS-MPI подключают `sal.h` из Visual Studio, а в MinGW его нет
build.sh	сборка всех этапов на Windows, Linux и macOS
plots.py	полный прогон экспериментов, CSV и графики в `results/`
docs/build_pdf.py	сборка PDF из `docs/*.md`
`results/`	CSV с замерами и PNG с графиками
`bin/`	собранные программы для Windows

Все четыре программы устроены одинаково, различаются только функцией разложения:

```
main ──► run(n) для каждого размера n из аргументов (или из списка по умолчанию)
           ├─ генерация A0 и b                          одинакова во всех этапах
           ├─ REPS раз:  A ← A0;  t ← wtime();
           │             block_lu(n, ALPHA, A);        ← единственное, что меняется
           │             t ← wtime() − t;  tmin ← min(tmin, t)
           ├─ lu_solve:  x из L·U·x = b
           └─ печать строки CSV:  n, alpha, tmin, err, res
```

2. Общие соглашения

2.1. Хранение матрицы

Матрица $n \times n$ лежит в одном массиве `double` по строкам (row-major):

$$a_{ij} \equiv \texttt{A[(size_t)i * n + j]}, \qquad 0 \le i, j < n.$$

В этапе 1 для этого заведён макрос `A(i, j)`, в этапах 2–3 — `ROW(i)`, указатель на начало строки. Приведение к `size_t` нужно потому, что произведение `i * n` в типе `int` переполняется уже при $n > 46\,340$.

Разложение выполняется **на месте**: после `block_lu` строго под диагональю лежат элементы L, на диагонали и над ней — элементы U:

$$A \longrightarrow \begin{pmatrix} u_{00} & u_{01} & \cdots & u_{0,n-1} \\ \ell_{10} & u_{11} & \cdots & u_{1,n-1} \\ \vdots & \ddots & \ddots & \vdots \\ \ell_{n-1,0} & \cdots & \ell_{n-1,n-2} & u_{n-1,n-1} \end{pmatrix}.$$

Единичная диагональ L ($\ell_{ii} = 1$) не хранится, а подразумевается. Поэтому в шаге 3 и в прямом ходе `lu_solve` нет деления на ℓ_{ii}.

2.2. Обозначения: пособие и код

В Алгоритме 37	В коде	Смысл
n	`n`	порядок матрицы
α	`ALPHA` (= 32), параметр `alpha`	размер малого блока
N	`N` = $\lceil n/\alpha \rceil$	число блоков по одной стороне
k	`k` (с нуля)	номер итерации
a	строки и столбцы `a0 … a1-1`	текущий малый блок
i, b	`i`, строки `i*alpha … END(i)-1`	малый блок внутри блочного столбца
j, c	`j`, столбцы `j*alpha … END(j)-1` (или `c0`, длина `m`)	малый блок внутри блочной строки
—	`s`, `l`, `q`	номера строки, строки-множителя и столбца внутри блоков
A_{22}, A'	та же часть массива `A`	A' сразу пишется поверх A_{22}

Основная операция любого шага записывается как $a_{sq} \mathrel{-}= a_{sl}\, a_{lq}$: `s` — обновляемая строка, `l` — строка блока a, из которой берётся множитель, `q` — столбец.

2.3. Константы и макросы

Имя	Значение	Где	Зачем
`ALPHA`	32	все этапы	размер блока. Три блока занимают 24 КБ и входят в L1; строка блока — 256 байт = ровно 8 AVX-векторов
`REPS`	3	все этапы	число повторов замера; в CSV пишется минимальное время
`END(i)`	$\min((i+1)\alpha,\, n)$	все этапы	конец блока i
`ROW(i)`	`A + i*n`	этапы 2–3	указатель на строку i
`LOCAL_ROW(i)`	`(i/size)*alpha*n`	этап 4	смещение блока i в локальном массиве владельца

2.4. Последний неполный блок

Пособие считает $\alpha = n/N$ целым (задача 2.4.4 предлагает обобщение). В коде $N = \lceil n/\alpha \rceil$, а граница блока вычисляется через `END`: например, при $n = 1000$ получается 31 полный блок и последний из $1000 - 992 = 8$

строк. Заключительное LU последнего блока из Алгоритма 37 — это просто итерация $k = N - 1$, на которой циклы шагов 2–4 пусты.

3. Одна итерация: от формул к коду

3.1. Четыре шага

Из разложения

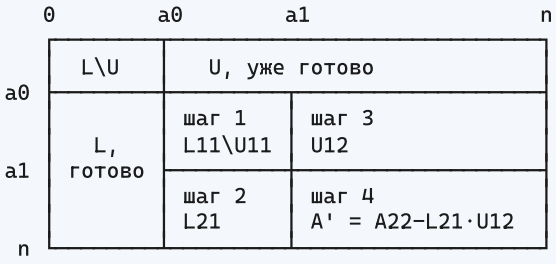
$$A = \begin{pmatrix} L_{11} & 0 \\ L_{21} & E \end{pmatrix} \begin{pmatrix} E & 0 \\ 0 & A' \end{pmatrix} \begin{pmatrix} U_{11} & U_{12} \\ 0 & E \end{pmatrix}$$

следует:

Шаг	Равенство	Поэлементно	Внутренний цикл
1	$A_{11} = L_{11}U_{11}$	$\ell_{sl} = a_{sl}/u_{ll};\ a_{sq} \mathrel{-}= \ell_{sl}u_{lq},\ q > l$	вдоль строки s блока a
2	$A_{21} = L_{21}U_{11}$	$\ell_{sl} = a_{sl}/u_{ll};\ a_{sq} \mathrel{-}= \ell_{sl}u_{lq},\ q > l$	вдоль строки s ниже блока
3	$A_{12} = L_{11}U_{12}$	$a_{sq} \mathrel{-}= \ell_{sl}u_{lq},\ s > l$	вдоль строки s правее блока
4	$A' = A_{22} - L_{21}U_{12}$	$a_{sq} \mathrel{-}= \ell_{sl}u_{lq}$	вдоль строки s правее и ниже блока

Шаги 1 и 2 имеют одинаковое тело, различаются только диапазон строк s. То же для шагов 3 и 4.

3.2. Где что лежит во время итерации



Итерация пишет в строки и столбцы от `a0` до `n`, всё левее и выше уже окончательно.

3.3. Стоимость шагов

Шаг	Флопов на итерации (m — строк ниже блока)	Всего за разложение	Доля при $n = 3000$
1. LU блока	$\frac{2}{3}\alpha^3$	$\frac{2}{3}n\alpha^2$	0,01%
2. L_{21}	$m\,\alpha^2$	$\approx \frac{1}{2}n^2\alpha$	0,8%
3. U_{12}	$m\,\alpha^2$	$\approx \frac{1}{2}n^2\alpha$	0,8%
4. мм.	$2\,m^2\alpha$	$\approx \frac{2}{3}n^3 - n^2\alpha$	98,4%
итого		$\frac{2}{3}n^3$	

3.4. Шаги в каждой программе

Шаг	Этап 1	Этап 2	Этап 3	Этап 4
1	строки 45–51	строки 51–56	строки 55–60, последовательно	строки 41–49, у владельца
2	строки 53–60	строки 58–64	строки 62–69, по блокам	строки 73–81, свои строки
3	строки 62–67	строки 66–72	строки 71–78, по блокам	строки 50–60, у владельца
4	строки 69–75	строки 74–81	строки 80–88, по парам блоков	строки 82–90, свои строки
обмен	—	—	неявный барьер после циклов	`MPI_Bcast`, строка 67

4. Этап 1 — `stage1_seq.c`

4.1. Таймер

stage1_seq.c · строки 14–31

```c
#ifdef _WIN32
#include <windows.h>
static double wtime(void)
{
    LARGE_INTEGER f, c;
    QueryPerformanceFrequency(&f);
    QueryPerformanceCounter(&c);
    return (double)c.QuadPart / (double)f.QuadPart;
}
#else
#include <time.h>
static double wtime(void)
{
    struct timespec t;
    clock_gettime(CLOCK_MONOTONIC, &t);
    return (double)t.tv_sec + 1e-9 * (double)t.tv_nsec;
}
#endif
```

На Windows время берётся из `QueryPerformanceCounter`: счётчик высокого разрешения (около 100 нс). На Linux и macOS — `clock_gettime(CLOCK_MONOTONIC)`: монотонные часы, которые не прыгают при переводе системного времени. `omp_get_wtime` не используется: в MinGW GCC 9.2 его разрешение около 1 мс.

4.2. Разложение `block_lu`

stage1_seq.c · строки 33–77

```c
/* A хранится по строкам: ниже диагонали — L, на диагонали и выше — U. */
#define A(i, j) A[(size_t)(i) * n + (j)]
/* Граница блока с номером i; последний блок может быть короче (задача 2.4.4). */
#define END(i) ((i) * alpha + alpha < n ? (i) * alpha + alpha : n)

void block_lu(int n, int alpha, double *A)
{
    int N = (n + alpha - 1) / alpha;

    for (int k = 0; k < N; k++) {
        int a0 = k * alpha, a1 = END(k);

        /* 1. A(a,a) = L(a,a) U(a,a): Алгоритм 30 со строчным saxpy */
        for (int l = a0; l < a1; l++)
            for (int s = l + 1; s < a1; s++) {
                A(s, l) /= A(l, l);
                for (int q = l + 1; q < a1; q++)
                    A(s, q) -= A(s, l) * A(l, q);
            }
```

```
52
53          /* 2. L(b,a) U(a,a) = A(b,a): для каждой строки xU = b (задача 2.1.3) */
54          for (int i = k + 1; i < N; i++)
55              for (int s = i * alpha; s < END(i); s++)
56                  for (int l = a0; l < a1; l++) {
57                      A(s, l) /= A(l, l);
58                      for (int q = l + 1; q < a1; q++)
59                          A(s, q) -= A(s, l) * A(l, q);
60                  }
61
62          /* 3. L(a,a) U(a,c) = A(a,c): Алгоритм 27 для каждого столбца блока */
63          for (int j = k + 1; j < N; j++)
64              for (int l = a0; l < a1; l++)
65                  for (int s = l + 1; s < a1; s++)
66                      for (int q = j * alpha; q < END(j); q++)
67                          A(s, q) -= A(s, l) * A(l, q);
68
69          /* 4. A'(b,c) = A(b,c) - L(b,a) U(a,c): основная операция */
70          for (int i = k + 1; i < N; i++)
71              for (int j = k + 1; j < N; j++)
72                  for (int s = i * alpha; s < END(i); s++)
73                      for (int l = a0; l < a1; l++)
74                          for (int q = j * alpha; q < END(j); q++)
75                              A(s, q) -= A(s, l) * A(l, q);
76      }
77  }
```

Разбор по строкам:

- **34, 36.** Макрос `A(i, j)` разворачивается в элемент массива `A`. Имя функционального макроса и имя указателя совпадают намеренно: `A(s, q)` читается как в пособии, а `A[...]` внутри макроса повторно не раскрывается. `END(i)` — конец блока с учётом неполного последнего.
- **40, 42–43.** Число блоков N и внешний цикл по итерациям; `a0`, `a1` — границы малого блока a.
- **45–51, шаг 1.** Строка l готова, для каждой строки $s > l$ блока вычисляется ℓ_{sl} и вычитается строка l. Это Алгоритм 30 со строчным saxpy (kij-форма).
- **53–60, шаг 2.** То же тело, но s пробегает строки ниже блока, а l — весь блок: для каждой строки решается система $xU = b$ (задача 2.1.3).
- **62–67, шаг 3.** Алгоритм 27 для каждого столбца блока c, с внутренним циклом вдоль строки. Деления нет: диагональ L единичная.
- **69–75, шаг 4.** Двойной цикл по малым блокам i, j и произведение блоков внутри — как в пособии.

4.3. Решение системы `lu_solve`

stage1_seq.c · строки 79–88

```
79  /* Решение L U x = b (x на входе содержит b) */
80  static void lu_solve(int n, const double *A, double *x)
81  {
82      for (int i = 0; i < n; i++)
83          for (int j = 0; j < i; j++) x[i] -= A(i, j) * x[j];
84      for (int i = n - 1; i >= 0; i--) {
85          for (int j = i + 1; j < n; j++) x[i] -= A(i, j) * x[j];
86          x[i] /= A(i, i);
87      }
88  }
```

Две подстановки на месте в массиве `x`, куда заранее скопирован `b`:

$$y_i = b_i - \sum_{j<i} \ell_{ij}\, y_j \quad (i = 0, 1, \ldots), \qquad x_i = \frac{1}{u_{ii}} \Big(y_i - \sum_{j>i} u_{ij}\, x_j \Big) \quad (i = n-1, \ldots, 0).$$

Прямой ход не делит: $\ell_{ii} = 1$. Стоимость — $2n^2$ флопов, время решения в замер не входит.

4.4. Эксперимент `run`

stage1_seq.c · строки 90–129

```c
static void run(int n)
{
    double *A0 = malloc((size_t)n * n * sizeof(double));
    double *A  = malloc((size_t)n * n * sizeof(double));
    double *b  = malloc(n * sizeof(double));
    double *x  = malloc(n * sizeof(double));

    srand(12345);
    for (size_t i = 0; i < (size_t)n * n; i++) A0[i] = (double)rand() / RAND_MAX - 0.5;
    /* Сдвиг диагонали не гарантирует строгого диагонального преобладания. */
    for (int i = 0; i < n; i++) A0[(size_t)i * n + i] += n / 4.0;
    for (int i = 0; i < n; i++) {           /* b = A x*, x* = (1,...,1) */
        double s = 0;
        for (int j = 0; j < n; j++) s += A0[(size_t)i * n + j];
        b[i] = s;
    }

    double tmin = 1e30;
    for (int rep = 0; rep < REPS; rep++) {
        for (size_t i = 0; i < (size_t)n * n; i++) A[i] = A0[i];
        double t = wtime();
        block_lu(n, ALPHA, A);
        t = wtime() - t;
        if (t < tmin) tmin = t;
    }

    for (int i = 0; i < n; i++) x[i] = b[i];
    lu_solve(n, A, x);
    double err = 0, res = 0, bn = 0;
    for (int i = 0; i < n; i++) {
        err = fmax(err, fabs(x[i] - 1.0));
        double s = -b[i];
        for (int j = 0; j < n; j++) s += A0[(size_t)i * n + j] * x[j];
        res = fmax(res, fabs(s));
        bn = fmax(bn, fabs(b[i]));
    }
    printf("%d,%d,%.6f,%.3e,%.3e\n", n, ALPHA, tmin, err, res / bn);
    fflush(stdout);
    free(A0); free(A); free(b); free(x);
}
```

- **97–100. Тестовая матрица.** Элементы равномерно распределены на $[-0.5,\ 0.5]$, к диагонали прибавлено $n/4$. Сдвиг делает ведущие элементы далёкими от нуля, и LU без выбора ведущего элемента устойчиво. `srand(12345)` даёт одну и ту же матрицу во всех этапах и при любом числе потоков и процессов.
- **101–105. Правая часть.** $b_i = \sum_j a_{ij}$, то есть $b = Ax^*$ при $x^* = (1, \dots, 1)^T$. Точное решение известно заранее, и погрешность можно посчитать напрямую.
- **107–114. Замер.** Перед каждым повтором `A` восстанавливается из `A0` (копирование в замер не входит). В CSV идёт минимальное из `REPS` времён: оно меньше всего искажено фоновыми процессами и прогревом кэша.
- **116–125. Метрики.** `err` $= \|x - x^*\|_\infty$ — прямая погрешность, `res` $= \|A_0 x - b\|_\infty / \|b\|_\infty$ — относительная невязка.
- **Память.** Две матрицы `A0` и `A` : $2n^2$ чисел `double`, при $n = 3000$ — 144 МБ.

4.5. `main`

stage1_seq.c · строки 131–140

```c
int main(int argc, char **argv)
{
    static const int def[] = {100, 250, 500, 750, 1000, 1500, 2000, 2500, 3000};
    printf("n,alpha,time,err,res\n");
```

```
135     if (argc > 1)
136         for (int i = 1; i < argc; i++) run(atoi(argv[i]));
137     else
138         for (size_t i = 0; i < sizeof def / sizeof *def; i++) run(def[i]);
139     return 0;
140 }
```

Без аргументов перебираются размеры из массива `def`, иначе каждый аргумент — размер матрицы. Аргументы не проверяются: `atoi` на мусоре вернёт 0, и программа напечатает бессмысленную строку для $n = 0$. `fflush(stdout)` в `run` нужен, чтобы при перенаправлении в файл строки появлялись сразу, а не по завершении программы.

5. Этап 2 — `stage2_vec.c`

Алгоритм и порядок операций — как в этапе 1. Изменена запись: вся арифметика вынесена в одну функцию `saxpy` с `restrict`-указателями. Таймер, `lu_solve`, `run` и `main` совпадают с этапом 1.

5.1. `saxpy`

stage2_vec.c · строки 33–38

```
33 /* y(0:m) -= s * x(0:m) — единственный векторизуемый цикл алгоритма */
34 static inline void saxpy(int m, double s, const double *restrict x, double *restrict y)
35 {
36     for (int q = 0; q < m; q++)
37         y[q] -= s * x[q];
38 }
```

Цикл идёт по соседним элементам строки, а `restrict` обещает компилятору, что `x` и `y` не перекрываются. Обещание верно: это разные строки матрицы. Множитель `s` передан по значению и не меняется в цикле, поэтому его можно размножить на все дорожки векторного регистра. За одну векторную FMA обрабатываются 4 числа. `static inline` позволяет компилятору встроить цикл во все четыре шага, без вызова функции (проверено по ассемблерному выводу).

5.2. `block_lu`

stage2_vec.c · строки 40–83

```
40 #define ROW(i) (A + (size_t)(i) * n)
41 #define END(i) ((i) * alpha + alpha < n ? (i) * alpha + alpha : n)
42
43 /* A (n x n, по строкам) заменяется на LU без выбора ведущего элемента. */
44 void block_lu(int n, int alpha, double *A)
45 {
46     int N = (n + alpha - 1) / alpha;
47
48     for (int k = 0; k < N; k++) {
49         int a0 = k * alpha, a1 = END(k);
50
51         /* 1. A(a,a) = L(a,a) U(a,a) */
52         for (int l = a0; l < a1; l++)
53             for (int s = l + 1; s < a1; s++) {
54                 ROW(s)[l] /= ROW(l)[l];
55                 saxpy(a1 - l - 1, ROW(s)[l], ROW(l) + l + 1, ROW(s) + l + 1);
56             }
57
58         /* 2. L(b,a) U(a,a) = A(b,a) */
59         for (int i = k + 1; i < N; i++)
60             for (int s = i * alpha; s < END(i); s++)
61                 for (int l = a0; l < a1; l++) {
62                     ROW(s)[l] /= ROW(l)[l];
63                     saxpy(a1 - l - 1, ROW(s)[l], ROW(l) + l + 1, ROW(s) + l + 1);
64                 }
```

```
65
66          /* 3. L(a,a) U(a,c) = A(a,c) */
67          for (int j = k + 1; j < N; j++) {
68              int c0 = j * alpha, m = END(j) - c0;
69              for (int l = a0; l < a1; l++)
70                  for (int s = l + 1; s < a1; s++)
71                      saxpy(m, ROW(s)[l], ROW(l) + c0, ROW(s) + c0);
72          }
73
74          /* 4. A'(b,c) = A(b,c) - L(b,a) U(a,c) */
75          for (int i = k + 1; i < N; i++)
76              for (int j = k + 1; j < N; j++) {
77                  int c0 = j * alpha, m = END(j) - c0;
78                  for (int s = i * alpha; s < END(i); s++)
79                      for (int l = a0; l < a1; l++)
80                          saxpy(m, ROW(s)[l], ROW(l) + c0, ROW(s) + c0);
81              }
82      }
83  }
```

Те же четыре шага. Адреса строк вычисляет `ROW`: `ROW(l) + c0` — начало блока $U(a,c)$ в строке l, `ROW(s) + c0` — начало блока $A(b,c)$ в строке s. Длина `m` на шагах 3 и 4 равна α (кроме неполного блока), поэтому цикл `saxpy` выполняет ровно 8 векторных итераций.

5.3. Две сборки одного файла

Программа	Флаги	Что получается
`stage2_novec`	`-O2 -fno-tree-vectorize`	скалярный код — база для сравнения
`stage2_vec`	`-O3 -ftree-vectorize -mavx2 -mfma`	цикл `saxpy` на AVX2 с FMA

На ARM (Apple M*, aarch64) `build.sh` не передаёт `-mavx2 -mfma`: векторы NEON по 2 числа и FMA там есть в базовом наборе команд. С FMA результат `stage2_vec` может отличаться от остальных этапов в последних знаках: ℓu не округляется перед вычитанием.

6. Этап 3 — `stage3_omp.c`

Код этапа 2, в котором циклы шагов 2–4 помечены `#pragma omp parallel for schedule(static)`. Итерации делит сама OpenMP; номер потока в коде нигде не используется.

6.1. `block_lu`

stage3_omp.c · строки 42–90

```
42  #define ROW(i) (A + (size_t)(i) * n)
43  #define END(i) ((i) * alpha + alpha < n ? (i) * alpha + alpha : n)
44
45  /* A (n x n, по строкам) заменяется на LU без выбора ведущего элемента.
46   * Шаг 1 последователен: это единственный шаг, где строки зависят друг от друга.
47   * В шагах 2-4 малые блоки независимы; каждый цикл завершается неявным барьером. */
48  void block_lu(int n, int alpha, double *A)
49  {
50      int N = (n + alpha - 1) / alpha;
51
52      for (int k = 0; k < N; k++) {
53          int a0 = k * alpha, a1 = END(k);
54
55          /* 1. A(a,a) = L(a,a) U(a,a) */
56          for (int l = a0; l < a1; l++)
57              for (int s = l + 1; s < a1; s++) {
58                  ROW(s)[l] /= ROW(l)[l];
59                  saxpy(a1 - l - 1, ROW(s)[l], ROW(l) + l + 1, ROW(s) + l + 1);
60              }
61
```

```
62          /* 2. L(b,a) U(a,a) = A(b,a): блоки блочного столбца независимы */
63          #pragma omp parallel for schedule(static)
64          for (int i = k + 1; i < N; i++)
65              for (int s = i * alpha; s < END(i); s++)
66                  for (int l = a0; l < a1; l++) {
67                      ROW(s)[l] /= ROW(l)[l];
68                      saxpy(a1 - l - 1, ROW(s)[l], ROW(l) + l + 1, ROW(s) + l + 1);
69                  }
70
71          /* 3. L(a,a) U(a,c) = A(a,c): блоки блочной строки независимы */
72          #pragma omp parallel for schedule(static)
73          for (int j = k + 1; j < N; j++) {
74              int c0 = j * alpha, m = END(j) - c0;
75              for (int l = a0; l < a1; l++)
76                  for (int s = l + 1; s < a1; s++)
77                      saxpy(m, ROW(s)[l], ROW(l) + c0, ROW(s) + c0);
78          }
79
80          /* 4. A'(b,c) = A(b,c) - L(b,a) U(a,c): пары блоков (i, j) независимы */
81          #pragma omp parallel for collapse(2) schedule(static)
82          for (int i = k + 1; i < N; i++)
83              for (int j = k + 1; j < N; j++) {
84                  int c0 = j * alpha, m = END(j) - c0;
85                  for (int s = i * alpha; s < END(i); s++)
86                      for (int l = a0; l < a1; l++)
87                          saxpy(m, ROW(s)[l], ROW(l) + c0, ROW(s) + c0);
88              }
89      }
90  }
```

- **55–60, шаг 1.** Последовательно: строки блока зависят друг от друга, а работы здесь $0{,}01\%$.
- **63–69, шаг 2.** Итерация i пишет только строки своего блока в столбцах a, читает блок $U(a,a)$, который на этом шаге не меняется.
- **72–78, шаг 3.** Итерация j пишет только столбцы своего блока. Внутри блока строки зависимы, поэтому параллелятся именно блоки, а не строки.
- **81–88, шаг 4.** `collapse(2)` превращает двойной цикл в один из $(N-k-1)^2$ итераций: каждая итерация пишет свой блок $A(b,c)$ и читает $L(b,a), U(a,c)$, куда на этом шаге никто не пишет.

6.2. Сводка параллельных циклов

Строка	Шаг	Что делится	Итераций
63	2	блоки блочного столбца	$N-k-1$
72	3	блоки блочной строки	$N-k-1$
81	4	пары блоков	$(N-k-1)^2$

На итерацию приходится 3 параллельные области, при $n = 3000$ — около 280 fork–join за разложение. Каждая область заканчивается неявным барьером, поэтому шаги идут строго друг за другом, как в последовательном коде.

Переменные, объявленные внутри параллельного цикла (`c0`, `m`, `s`, `l`), приватны для потока. Аргументы функции (`n`, `alpha`, `A`), а также `k`, `a0`, `a1` общие, но внутри параллельных циклов только читаются. Счётчики циклов, в том числе обоих циклов под `collapse(2)`, OpenMP делает приватными сама.

Порядок вычитаний над каждым элементом не зависит от распределения по потокам, поэтому этап 3 даёт **побитово тот же результат**, что этап 1, при любом числе потоков. Это проверено на x86; на ARM компилятор по умолчанию сливает умножение с вычитанием в FMA, и последние знаки могут разойтись.

В CSV поле `threads` берётся из `omp_get_max_threads()`, то есть из `OMP_NUM_THREADS`.

7. Этап 4 — `stage4_mpi.c`

Каждый процесс хранит только свои блочные строки. На итерации k владелец блочной строки k выполняет шаги 1 и 3 и рассылает её, а все процессы выполняют шаги 2 и 4 над своими строками.

7.1. Распределение

stage4_mpi.c · строки 14–21

```c
#define ALPHA 32
#define REPS  3
static int rank, size;

#define END(i) ((i) * alpha + alpha < n ? (i) * alpha + alpha : n)
/* Начало строк блока i в локальном массиве владельца: все предшествующие
 * блоки этого процесса полные, короче может быть только последний блок матрицы. */
#define LOCAL_ROW(i) ((size_t)((i) / size) * alpha * n)
```

`rank` и `size` — номер процесса и их число, глобальные для краткости. Блок i принадлежит процессу $i \bmod p$. Внутри процесса блоки лежат подряд, и `LOCAL_ROW(i)` даёт смещение начала блока: например, при $p = 3$ у процесса 1 блоки 1, 4, 7 находятся на локальных местах 0, 1, 2.

7.2. `block_lu_mpi`

stage4_mpi.c · строки 29–94

```c
/* A — локальные блочные строки этого процесса, по n элементов в строке. */
void block_lu_mpi(int n, int alpha, double *A)
{
    int N = (n + alpha - 1) / alpha;
    double *W = malloc((size_t)alpha * n * sizeof(double));   /* блочная строка k */

    for (int k = 0; k < N; k++) {
        int a0 = k * alpha, w = END(k) - a0, t = n - a0;      /* W: w строк по t чисел */
        int owner = k % size;

        if (rank == owner) {
            double *Ak = A + LOCAL_ROW(k);
            /* 1. A(a,a) = L(a,a) U(a,a) */
            for (int l = 0; l < w; l++) {
                const double *rl = Ak + (size_t)l * n + a0;
                for (int s = l + 1; s < w; s++) {
                    double *rs = Ak + (size_t)s * n + a0;
                    rs[l] /= rl[l];
                    saxpy(w - l - 1, rs[l], rl + l + 1, rs + l + 1);
                }
            }
            /* 3. L(a,a) U(a,c) = A(a,c) */
            for (int j = k + 1; j < N; j++) {
                int c0 = j * alpha, m = END(j) - c0;
                for (int l = 0; l < w; l++) {
                    const double *rl = Ak + (size_t)l * n;
                    for (int s = l + 1; s < w; s++) {
                        double *rs = Ak + (size_t)s * n;
                        saxpy(m, rs[a0 + l], rl + c0, rs + c0);
                    }
                }
            }
            for (int s = 0; s < w; s++)
                for (int q = 0; q < t; q++)
                    W[(size_t)s * t + q] = Ak[(size_t)s * n + a0 + q];
        }
        /* После последнего блока строк ниже нет: рассылать некому и незачем. */
        if (k == N - 1) break;
        MPI_Bcast(W, w * t, MPI_DOUBLE, owner, MPI_COMM_WORLD);

        /* Первый свой блок ниже k: i >= k + 1, i % size == rank */
        for (int i = k + 1 + ((rank - k - 1) % size + size) % size; i < N; i += size) {
```

```
71              double *Ai = A + LOCAL_ROW(i);
72              int h = END(i) - i * alpha;
73              /* 2. L(b,a) U(a,a) = A(b,a) */
74              for (int s = 0; s < h; s++) {
75                  double *rs = Ai + (size_t)s * n + a0;
76                  for (int l = 0; l < w; l++) {
77                      const double *ul = W + (size_t)l * t;
78                      rs[l] /= ul[l];
79                      saxpy(w - l - 1, rs[l], ul + l + 1, rs + l + 1);
80                  }
81              }
82              /* 4. A'(b,c) = A(b,c) - L(b,a) U(a,c) */
83              for (int j = k + 1; j < N; j++) {
84                  int c0 = j * alpha, m = END(j) - c0;
85                  for (int s = 0; s < h; s++) {
86                      double *rs = Ai + (size_t)s * n;
87                      for (int l = 0; l < w; l++)
88                          saxpy(m, rs[a0 + l], W + (size_t)l * t + c0 - a0, rs + c0);
89                  }
90              }
91          }
92      }
93      free(W);
94  }
```

- **33, 36.** `W` — буфер блочной строки k: `w` строк по `t` чисел, начиная с диагонального блока, без неполных строк и лишних столбцов. Память — αn чисел.
- **39–49, шаг 1.** Владелец раскладывает блок. `Ak` — начало блока k в локальном массиве, `rl` и `rs` — строки со сдвигом на `a0`.
- **50–60, шаг 3.** Владелец находит блоки $U(a,c)$; `rs[a0 + l]` — множитель ℓ_{sl}.
- **61–63.** Упаковка блочной строки в `W`: в локальном массиве шаг строки `n`, а `MPI_Bcast` нужен непрерывный участок.
- **66–67.** Единственный обмен алгоритма. После последнего блока строк ниже нет, рассылать незачем.
- **70.** Первый блок этого процесса ниже k, далее шаг `size`. Двойной остаток нужен, потому что `%` в C для отрицательных чисел даёт отрицательный результат.
- **73–81, шаг 2.** Для каждой строки своего блока решается $xU = b$ с матрицей U_{11} из `W`.
- **82–90, шаг 4.** По малым блокам c: `W + l*t + c0 - a0` — начало блока $U(a,c)$ в строке l буфера.

7.3. Раздача и сбор строк

stage4_mpi.c · строки 96–128

```
96  /* Раздать блочные строки A0 (процесс 0) по локальным массивам A. Вне замера. */
97  static void scatter_rows(int n, int alpha, const double *A0, double *A)
98  {
99      int N = (n + alpha - 1) / alpha;
100     for (int i = 0; i < N; i++) {
101         int h = END(i) - i * alpha, owner = i % size;
102         if (rank == 0 && owner == 0) {
103             for (size_t e = 0; e < (size_t)h * n; e++)
104                 A[LOCAL_ROW(i) + e] = A0[(size_t)i * alpha * n + e];
105         } else if (rank == 0) {
106             MPI_Send(A0 + (size_t)i * alpha * n, h * n, MPI_DOUBLE, owner, i, MPI_COMM_WORLD);
107         } else if (rank == owner) {
108             MPI_Recv(A + LOCAL_ROW(i), h * n, MPI_DOUBLE, 0, i, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
109         }
110     }
111 }
112
113 /* Собрать разложение на процессе 0, чтобы проверить решение. Вне замера. */
114 static void gather_rows(int n, int alpha, const double *A, double *LU)
115 {
116     int N = (n + alpha - 1) / alpha;
117     for (int i = 0; i < N; i++) {
118         int h = END(i) - i * alpha, owner = i % size;
```

```
119         if (rank == 0 && owner == 0) {
120             for (size_t e = 0; e < (size_t)h * n; e++)
121                 LU[(size_t)i * alpha * n + e] = A[LOCAL_ROW(i) + e];
122         } else if (rank == 0) {
123             MPI_Recv(LU + (size_t)i * alpha * n, h * n, MPI_DOUBLE, owner, i, MPI_COMM_WORLD,
MPI_STATUS_IGNORE);
124         } else if (rank == owner) {
125             MPI_Send(A + LOCAL_ROW(i), h * n, MPI_DOUBLE, 0, i, MPI_COMM_WORLD);
126         }
127     }
128 }
```

Обе функции идут по блокам и работают только с процессом 0: он посылает или принимает блочную строку у её владельца, свои блоки копирует локально. Тегом служит номер блока. Взаимной блокировки быть не может, потому что все обмены идут между процессом 0 и одним другим процессом в едином порядке. Обе функции вне замера времени.

7.4. Эксперимент `run`

stage4_mpi.c · строки 130–161

```
130 static void run(int n)
131 {
132     int alpha = ALPHA, N = (n + alpha - 1) / alpha, rows = 0;
133     for (int i = rank; i < N; i += size) rows += END(i) - i * alpha;
134     /* При числе процессов больше N у процесса может не быть строк. */
135     double *A = malloc(((size_t)rows * n + 1) * sizeof(double));
136     double *A0 = NULL, *b = NULL;
137 
138     if (rank == 0) {
139         A0 = malloc((size_t)n * n * sizeof(double));
140         b = malloc(n * sizeof(double));
141         srand(12345);
142         for (size_t i = 0; i < (size_t)n * n; i++) A0[i] = (double)rand() / RAND_MAX - 0.5;
143         for (int i = 0; i < n; i++) A0[(size_t)i * n + i] += n / 4.0;
144         for (int i = 0; i < n; i++) {
145             double s = 0;
146             for (int j = 0; j < n; j++) s += A0[(size_t)i * n + j];
147             b[i] = s;
148         }
149     }
150 
151     double tmin = 1e30;
152     for (int rep = 0; rep < REPS; rep++) {
153         scatter_rows(n, alpha, A0, A);
154         MPI_Barrier(MPI_COMM_WORLD);
155         double t = MPI_Wtime();
156         block_lu_mpi(n, alpha, A);
157         /* Конечный барьер входит во время: ждём самый медленный процесс. */
158         MPI_Barrier(MPI_COMM_WORLD);
159         t = MPI_Wtime() - t;
160         if (t < tmin) tmin = t;
161     }
```

- **133, 135.** Размер локального массива. Единица добавлена, чтобы `malloc(0)` не вернул `NULL`, когда процессов больше, чем блоков (при очень малых n).
- **138–149.** Матрицу генерирует только процесс 0; `A0` и `b` есть только у него.
- **153.** Раздача строк перед каждым повтором — вне замера.
- **154, 158.** `MPI_Barrier` перед замером выравнивает старт процессов, после — гарантирует, что в время попал самый медленный процесс.

Проверка решения (строки 163–187) — те же подстановки и те же метрики, что в этапе 1, на процессе 0, по разложению, собранному `gather_rows`. `main` дополнительно вызывает `MPI_Init`, `MPI_Comm_rank`, `MPI_Comm_size` и `MPI_Finalize`; заголовок CSV печатает только процесс 0.

7.5. Вызовы MPI

Вызов	Где	Сколько данных
`MPI_Send`, `MPI_Recv`	`scatter_rows`, перед повтором	n^2 чисел, вне замера
`MPI_Barrier`	`run`, вокруг замера	—
`MPI_Bcast`	`block_lu_mpi`, строка 67	$w \cdot t$ чисел на итерацию, $\approx \frac{1}{2}n^2$ всего
`MPI_Send`, `MPI_Recv`	`gather_rows`, после повторов	n^2 чисел, вне замера

При $n = 3000$ за разложение рассылается ≈ 36 МБ. Память процесса — n^2/p чисел строк плюс αn буфера `W`: при $n = 3000$ и $p = 4$ около 19 МБ. Процесс 0 ещё держит `A0` и `LU` для проверки.

8. Сравнение реализаций

	Этап 1	Этап 2	Этап 3	Этап 4
данные	`A`	`A`	`A`, общая	свои блочные строки + `W`
запись	макрос `A(i, j)`	`saxpy` + `restrict`	`saxpy` + `restrict`	`saxpy` + `restrict`
шаг 1	последовательно	последовательно, SIMD	последовательно	у владельца
шаг 2	последовательно	SIMD	потоки по блокам	свои строки
шаг 3	последовательно	SIMD	потоки по блокам	у владельца
шаг 4	последовательно	SIMD	потоки по парам блоков	свои строки
синхронизация	—	—	неявный барьер после циклов	`MPI_Bcast` на итерации
результат	эталон	`novec` = эталон, `vec` отличается в последних знаках (FMA)	= эталон	= эталон

9. Вспомогательные скрипты

9.1. `build.sh`

Определяет ОС через `uname -s` и архитектуру через `uname -m` и собирает пять программ.

	Windows (Git Bash)	Linux	macOS
компилятор	`gcc` (MinGW)	`cc` или `$CC`	`cc` (Apple clang) или `$CC`
общие флаги	`-std=gnu99 -msse2 -mfpmath=sse`	`-std=gnu99` , `-lm`	`-std=gnu99` , `-lm`
без векторизации	`-O2 -fno-tree-vectorize`	то же; у clang ещё `-fno-slp-vectorize`	то же
векторизация	`-O3 -ftree-vectorize -mavx2 -mfma`	на x86 — то же, на ARM без `-mavx2 -mfma`	то же
OpenMP	`-fopenmp`	`-fopenmp`	Apple clang: `-Xpreprocessor -fopenmp -lomp` + пути `libomp`
MPI	`gcc` + MS-MPI (`Lib/x86` , `include/sal.h`)	`mpicc`	`mpicc`
результат	`bin/*.exe`	`bin/*`	`bin/*`

`-msse2 -mfpmath=sse` нужны только 32-битному MinGW: без них он считает на x87 с 80-битными промежуточными результатами, и цифры не совпали бы с 64-битными системами.

9.2. `plots.py`

1. Этап 1 на размерах 100…3000 → `stage1_seq.csv` , график погрешности.
2. Обе сборки этапа 2 → `stage2_novec.csv` , `stage2_vec.csv` , график ускорения.
3. Этап 3 с `OMP_NUM_THREADS` = 1…8 на размерах 500…3000 → `stage3_omp_t{P}.csv` , два графика ускорения T_1/T_P.
4. Этап 4 через `mpiexec -n` 1…8 → `stage4_mpi_p{P}.csv` , два графика. Для Open MPI скрипт сам добавляет `--oversubscribe` .

Скрипт перезаписывает `results/` и не проверяет `err` и `res` .

9.3. `docs/build_pdf.py`

Собирает PDF из Markdown: формулы прячет от Markdown, рендерит их KaTeX в headless Chrome, ссылки между документами превращает во внутренние якоря, картинки встраивает в HTML. Листинги с пометкой `<!-- src: файл a-b -->` берёт прямо из исходников, подсвечивает и нумерует настоящими номерами строк. Если текст листинга в `.md` разошёлся с кодом, скрипт печатает предупреждение. Книги перечислены в словаре `BOOKS` : `block_lu` — теория и этапы, `code_reference` — этот справочник.

10. Вывод программ

Каждая программа печатает в `stdout` CSV; в MPI — только процесс 0.

Программа	Заголовок
`stage1_seq` , `stage2_novec` , `stage2_vec`	`n,alpha,time,err,res`
`stage3_omp`	`n,threads,alpha,time,err,res`
`stage4_mpi`	`procs,n,alpha,time,err,res`

`time` — минимальное время разложения из трёх повторов в секундах (без генерации, копирования и решения), `err` и `res` — метрики из раздела 4.4.

11. Частые вопросы

Почему нет выбора ведущего элемента? В Алгоритме 37 его нет, а тестовая матрица со сдвигом диагонали на $n/4$ даёт ведущие элементы, далёкие от нуля. Условия существования и устойчивости такого разложения выведены в 00_lu_theory.md. На произвольной матрице программа может поделить на ноль или потерять точность.

Почему цифры `err` и `res` на Linux/macOS другие? Матрицу строит `rand()`, а его алгоритм и `RAND_MAX` зависят от библиотеки C: в MSVCRT на Windows `RAND_MAX` = 32767, в glibc — $2^{31} - 1$. Матрицы получаются разными, но того же типа, и погрешности остаются того же порядка.

Почему этапы 1, 3, 4 дают ровно те же цифры, а `stage2_vec` — нет? Параллелизм меняет только то, *кто* считает элемент, а не порядок операций над ним. Векторизация с FMA меняет округление: $a - \ell u$ считается одной операцией.

Почему шаг 1 не параллелен, а внешний цикл по k нельзя параллелить? Строки блока зависят друг от друга, а итерация k работает с результатом итерации $k - 1$. На шаге 1 работы всего $0{,}01\%$.

Зачем в MPI буфер `W`, если строка уже есть у владельца? В локальном массиве у строк шаг `n`, и блочная строка не лежит непрерывно, а остальным процессам она нужна только на текущую итерацию.

Почему распределение циклическое, а не непрерывными кусками? На итерации k работают только блоки ниже k. При непрерывном распределении первые процессы быстро останутся без работы.

Как поменять размер блока? Изменить `#define ALPHA` в нужном файле и пересобрать. Остальной код от значения не зависит. Удобны значения, кратные 4: на AVX2 в цикле `saxpy` не остаётся скалярного «хвоста».

12. Шпаргалка: где что искать

Что	Этап 1	Этап 2	Этап 3	Этап 4
таймер	14–31	14–31	17–34	`MPI_Wtime`, 155–159
цикл по итерациям	42	48	52	35
шаг 1	45–51	51–56	55–60	41–49
шаг 2	53–60	58–64	62–69	73–81
шаг 3	62–67	66–72	71–78	50–60
шаг 4	69–75	74–81	80–88	82–90
решение $LUx = b$	`lu_solve`, 80–88	`lu_solve`, 85–93	`lu_solve`, 92–100	167–174
генерация матрицы	97–105	102–109	109–116	138–149
метрики	118–125	122–129	129–136	175–182
строка CSV	126	130	137	183